LTH Challenge 2025

Authors
Gustav Nilsson Gisleskog, Jonathan Cederlund, Julius Thunström,
Max Nilsson, Niklas Sandén

May 15, 2025

Summary

A - Wake Up Call

Problem

Given 2 sequences of numbers with length $N, M \leq 2 \cdot 10^5$. Which sequence have the greatest sum.

Solution

A - Wake Up Call

Problem

Given 2 sequences of numbers with length $N, M \leq 2 \cdot 10^5$. Which sequence have the greatest sum.

Solution

Sum up the sequences and check which is bigger. Print the corresponding text. *Button 1*, *Button 2*, *Oh no*

B - Monsters, Inc

Problem

There are N monsters in a row with strength requirements. Defeating monster i increases your strength by b_i. Given queries of starting strength, how many monsters can you defeat?

B - Monsters, Inc

Problem

There are N monsters in a row with strength requirements. Defeating monster i increases your strength by b_i. Given queries of starting strength, how many monsters can you defeat?

Solution 1 - 20p

Constraints: $N, Q \leq 100$

- Simulate each query - $\mathcal{O}(NQ)$

B - Monsters, Inc

Problem

There are N monsters in a row with strength requirements. Defeating monster i increases your strength by b_i. Given queries of starting strength, how many monsters can you defeat?

Solution 1 - 20p

Constraints: $N, Q \leq 100$

- Simulate each query - $\mathcal{O}(NQ)$

Solution 2 - 30p

Constraints: $b_i = 0$

- Calculate prefix sum of strength requirements.
- Binary search for each query - $\mathcal{O}(Q \log N)$

B - Monsters, Inc

Solution 3 - 100p

- Same as Solution 2 but reduce by accumulated strength as you are calculating the prefix sum - $\mathcal{O}(Q \log N)$

B - Monsters, Inc

Solution 3 - 100p

- Same as Solution 2 but reduce by accumulated strength as you are calculating the prefix sum - $\mathcal{O}(Q \log N)$
- Alternative solution: Sort the queries by starting strength (already done for you in the problem). Then go through the monsters one by one. Keep track of strength accumulated strength and pop queries while they would fail - $\mathcal{O}(N + Q)$

C - Age Guessing Dilemma

Problem

Interactively guess the number between 1 and 10^9. You will find out if your guess is too high or too low after each guess. Guessing too high results in B times higher penalty than guessing too low.

C - Age Guessing Dilemma

Problem

Interactively guess the number between 1 and 10^9. You will find out if your guess is too high or too low after each guess. Guessing too high results in B times higher penalty than guessing too low.

Solution 1 - 20p

Constraints: $B = 1$

- Binary search normally. $\approx \log_2 10^9$ penalty.

C - Age Guessing Dilemma

Solution 2 - 100p

- Make the right interval B times larger than the left interval.
- Always higher $\approx \log_{\frac{B+1}{B}}(10^9)$ penalty.
- Always lower $\approx B \log_{\frac{B+1}{1}}(10^9)$ penalty.
- Exercise for the reader: what is the optimal strategy?

D - Vacationtime

Problem

Given a connected graph with 2 types of directed edges A, B. Find the cheapest path from node 0 to node $A - 1$ where at least one connection is made over a edge of type B

D - Vacationtime

Problem

Given a connected graph with 2 types of directed edges A, B. Find the cheapest path from node 0 to node $A-1$ where at least one connection is made over a edge of type B

Solution 1 - 10p

Constraints: cost $a \to b = 1$, One edge of type B

- Find the shortest path from 0 to the start of edge B, Then find the shortest path from end of edge B to $A-1$
- cost $a \to b = 1 \implies$ use BFS to find the shortest path.

D - Vacationtime

Solution 2 - 30p

Constraints: One edge of type B

- Find the shortest path from 0 to the start of edge B, Then find the shortest path from end of edge B to $A - 1$

Solution 3 - 100p

Constraints: $A \leq 10^4, F \leq 10^5$

D - Vacationtime

Solution 2 - 30p

Constraints: One edge of type B

- Find the shortest path from 0 to the start of edge B, Then find the shortest path from end of edge B to $A - 1$
- Cost is no longer 1 but is guaranteed to be positive

Solution 3 - 100p

Constraints: $A \leq 10^4, F \leq 10^5$

D - Vacationtime

Solution 2 - 30p

Constraints: One edge of type B

- Find the shortest path from 0 to the start of edge B, Then find the shortest path from end of edge B to $A - 1$
- Cost is no longer 1 but is guaranteed to be positive
- Use Dijkstra to find the cheapest path.

Solution 3 - 100p

Constraints: $A \leq 10^4, F \leq 10^5$

D - Vacationtime

Solution 2 - 30p

Constraints: One edge of type B

- Find the shortest path from 0 to the start of edge B, Then find the shortest path from end of edge B to $A-1$
- Cost is no longer 1 but is guaranteed to be positive
- Use Dijkstra to find the cheapest path.

Solution 3 - 100p

Constraints: $A \leq 10^4, F \leq 10^5$

- At a given airport a_i either you have use an edge B or not.

D - Vacationtime

Solution 2 - 30p

Constraints: One edge of type B

- Find the shortest path from 0 to the start of edge B, Then find the shortest path from end of edge B to $A-1$
- Cost is no longer 1 but is guaranteed to be positive
- Use Dijkstra to find the cheapest path.

Solution 3 - 100p

Constraints: $A \leq 10^4, F \leq 10^5$

- At a given airport a_i either you have use an edge B or not.
- Split every node into 2 nodes (a_i, a_{A+i}). (Duplicate graph trick)

D - Vacationtime

Solution 2 - 30p

Constraints: One edge of type B

- Find the shortest path from 0 to the start of edge B, Then find the shortest path from end of edge B to $A-1$
- Cost is no longer 1 but is guaranteed to be positive
- Use Dijkstra to find the cheapest path.

Solution 3 - 100p

Constraints: $A \leq 10^4, F \leq 10^5$

- At a given airport a_i either you have use an edge B or not.
- Split every node into 2 nodes (a_i, a_{A+i}). (Duplicate graph trick)
- Edges of typ A moves between the first type of nodes while edges of type B takes you to the second set of nodes.

D - Vacationtime

Solution 2 - 30p

Constraints: One edge of type B

- Find the shortest path from 0 to the start of edge B, Then find the shortest path from end of edge B to $A-1$
- Cost is no longer 1 but is guaranteed to be positive
- Use Dijkstra to find the cheapest path.

Solution 3 - 100p

Constraints: $A \leq 10^4, F \leq 10^5$

- At a given airport a_i either you have use an edge B or not.
- Split every node into 2 nodes (a_i, a_{A+i}). (Duplicate graph trick)
- Edges of typ A moves between the first type of nodes while edges of type B takes you to the second set of nodes.
- Use Dijkstra to find the cheapest path.

E - Drowssap

Problem

Given the code for a password verification function and an output from it, find an input producing the output.

E - Drowssap

Problem

Given the code for a password verification function and an output from it, find an input producing the output.

Analysis

- Input space has size 2^{40} so iterating over all inputs is far too slow.

E - Drowssap

Problem

Given the code for a password verification function and an output from it, find an input producing the output.

Analysis

- Input space has size 2^{40} so iterating over all inputs is far too slow.
- If you have watched 3B1B's recent video on quantum computing you might think, "Ah, I'll just implement Grover's algorithm!". Good idea, would love to see someone solve it that way.

E - Drowssap

Problem

Given the code for a password verification function and an output from it, find an input producing the output.

Analysis

- Input space has size 2^{40} so iterating over all inputs is far too slow.
- If you have watched 3B1B's recent video on quantum computing you might think, "Ah, I'll just implement Grover's algorithm!". Good idea, would love to see someone solve it that way.
- We should instead read the code a bit more carefully...

E - Drowssap

Problem

Given the code for a password verification function and an output from it, find an input producing the output.

Solution 1

- Input is split into two halves, some bit operations are applied, some of them combining the two halves.

E - Drowssap

Problem

Given the code for a password verification function and an output from it, find an input producing the output.

Solution 1

- Input is split into two halves, some bit operations are applied, some of them combining the two halves.
- Key thing to notice about the bit operations is that bitwise OR behaves different compared to the other operations. Specifically, given the result of "a | 0xaaaaaa", we cannot find a **unique** input producing the output as bitwise OR forces some bits to be active.

E - Drowssap

Problem

Given the code for a password verification function and an output from it, find an input producing the output.

Solution 1

- Input is split into two halves, some bit operations are applied, some of them combining the two halves.
- Key thing to notice about the bit operations is that bitwise OR behaves different compared to the other operations. Specifically, given the result of "a | 0xaaaaaa", we cannot find a **unique** input producing the output as bitwise OR forces some bits to be active.
- Examining the bit pattern of 0xaaaaaa reveals it looks like "10101010101010101010". We see that only half of the bits are actually controlled by the input. This is true for both halves of the entire 40-bit input.

E - Drowssap

Problem

Given the code for a password verification function and an output from it, find an input producing the output.

Solution 1

- Examining the bit pattern of 0xaaaaaa reveals it looks like "101010101010101010". We see that only half of the bits are actually controlled by the input. This is true for both halves of the entire 40-bit input.

E - Drowssap

Problem

Given the code for a password verification function and an output from it, find an input producing the output.

Solution 1

- Examining the bit pattern of 0xaaaaaa reveals it looks like "101010101010101010". We see that only half of the bits are actually controlled by the input. This is true for both halves of the entire 40-bit input.
- We therefore only need to iterate over the bit positions where 0xaaaaaa has zeros. This search space is only 2^{20} which is fast to iterate over, even in Python.

E - Drowssap

Problem

Given the code for a password verification function and an output from it, find an input producing the output.

Solution 1

- Examining the bit pattern of 0xaaaaaa reveals it looks like "10101010101010101010". We see that only half of the bits are actually controlled by the input. This is true for both halves of the entire 40-bit input.
- We therefore only need to iterate over the bit positions where 0xaaaaaa has zeros. This search space is only 2^{20} which is fast to iterate over, even in Python.
- A consequence of this design is that since only 20 bits actually matter, guessing random 40-bit sequences has a very high probability of succeeding if you test enough of them...

E - Drowssap

Problem

Given the code for a password verification function and an output from it, find an input producing the output.

Solution 2

- b = (output := int(input())) & 0xfffff
- a = output » 20
- a = a ^ b
- b = rotate(b, 14)
- b = b ^ 0xbf83f
- b = b ^ a
- a = a ^ 0xc21f3
- a = rotate(a, 14)
- print((b « 20) | a)

F - Spare Key

Problem

N people have given out a number of spare keys. Given the order in which people go outside and lose their keys, find how many are permanently locked out after each step.

Solution 1 - 15p

F - Spare Key

Problem

N people have given out a number of spare keys. Given the order in which people go outside and lose their keys, find how many are permanently locked out after each step.

Solution 1 - 15p

Constraints: $N, M \leq 1000$

1. Build a directed graph. Each person is a node, and there is an edge from u to v if u holds a spare key for v.

F - Spare Key

Problem

N people have given out a number of spare keys. Given the order in which people go outside and lose their keys, find how many are permanently locked out after each step.

Solution 1 - 15p

Constraints: $N, M \leq 1000$

1. Build a directed graph. Each person is a node, and there is an edge from u to v if u holds a spare key for v.
2. In each time step, search with e.g. BFS/DFS from each person who has not left the house yet, to see who they can help get inside, who the next person can help get inside and so on.

F - Spare Key

Problem

N people have given out a number of spare keys. Given the order in which people go outside and lose their keys, find how many are permanently locked out after each step.

Solution 1 - 15p

Constraints: $N, M \leq 1000$

1. Build a directed graph. Each person is a node, and there is an edge from u to v if u holds a spare key for v.
2. In each time step, search with e.g. BFS/DFS from each person who has not left the house yet, to see who they can help get inside, who the next person can help get inside and so on.
3. We will do this N times, and each search takes $\mathcal{O}(N + M)$ time, resulting in $\mathcal{O}(N^2 + NM)$.

F - Spare Key

Solution 2 - 100p

Now $N, M \leq 5 \cdot 10^5$.

1. N is too big to do a big search every time.

F - Spare Key

Solution 2 - 100p

Now $N, M \leq 5 \cdot 10^5$.

1. N is too big to do a big search every time.
2. Consider the backwards problem: Everyone is locked outside in the beginning, then they magically enter their homes. How many people are locked out in each step?

F - Spare Key

Solution 2 - 100p

Now $N, M \leq 5 \cdot 10^5$.

1. N is too big to do a big search every time.
2. Consider the backwards problem: Everyone is locked outside in the beginning, then they magically enter their homes. How many people are locked out in each step?
3. When a person comes home, search from them that are still permanently locked out. Do not continue the search into people that we already know can be helped.

F - Spare Key

Solution 2 - 100p

Now $N, M \leq 5 \cdot 10^5$.

1. N is too big to do a big search every time.
2. Consider the backwards problem: Everyone is locked outside in the beginning, then they magically enter their homes. How many people are locked out in each step?
3. When a person comes home, search from them that are still permanently locked out. Do not continue the search into people that we already know can be helped.
4. Results in $\mathcal{O}(N + M)$, because each edge is traversed/checked exactly once.

G - Ice to Meet You

Problem

Find two students for which a non-educational event may happen or report that such an event cannot happen.

G - Ice to Meet You

Problem

Find two students for which a non-educational event may happen or report that such an event cannot happen.

Analysis

- The condition for two students matching, $ax_1 + by_1 = ax_2 + by_2$, basically just says that two students match if you can draw a line with rational slope between the points (x_1, y_1) and (x_2, y_2).

G - Ice to Meet You

Problem

Find two students for which a non-educational event may happen or report that such an event cannot happen.

Analysis

- The condition for two students matching, $ax_1 + by_1 = ax_2 + by_2$, basically just says that two students match if you can draw a line with rational slope between the points (x_1, y_1) and (x_2, y_2).
- This means that a non-educational event is the case where no other points except (x_1, y_1) and (x_2, y_2) lie on that same line. In mathematics, such a line is called an ordinary line and a point belonging to an ordinary line is called an ordinary point.

G - Ice to Meet You

Problem

Find two students for which a non-educational event may happen or report that such an event cannot happen.

Analysis

- The condition for two students matching, $ax_1 + by_1 = ax_2 + by_2$, basically just says that two students match if you can draw a line with rational slope between the points (x_1, y_1) and (x_2, y_2).
- This means that a non-educational event is the case where no other points except (x_1, y_1) and (x_2, y_2) lie on that same line. In mathematics, such a line is called an ordinary line and a point belonging to an ordinary line is called an ordinary point.
- A non-educational event cannot happen if all points are colinear. That is, all points lie on the same line. This is the Sylvester–Gallai theorem.

G - Ice to Meet You

Problem

Find two students for which a non-educational event may happen or report that such an event cannot happen.

Solution 1 - 10p

Constraints: $N \leq 1000$

- One $\mathcal{O}(n^2)$ solution is then to loop over all pairs of points and solve for the line passing between them. We can represent each line as the 3-tuple (A, B, C), representing the standard form $Ax + By = C$ (unique if ensuring A, B and C have no common factors and $A \geq 0$.

G - Ice to Meet You

Problem

Find two students for which a non-educational event may happen or report that such an event cannot happen.

Solution 1 - 10p

Constraints: $N \leq 1000$

- One $\mathcal{O}(n^2)$ solution is then to loop over all pairs of points and solve for the line passing between them. We can represent each line as the 3-tuple (A, B, C), representing the standard form $Ax + By = C$ (unique if ensuring A, B and C have no common factors and $A \geq 0$.
- Count how often each line appears and then at the end check if some line appears only once. If so, say which points formed it, otherwise say "learning is guaranteed!"

G - Ice to Meet You

Problem

Find two students for which a non-educational event may happen or report that such an event cannot happen.

Solution 2 - 100p

Constraints: $N \leq 10^6$

- Quadratic solution is now way too slow but faster ones exist.

G - Ice to Meet You

Problem

Find two students for which a non-educational event may happen or report that such an event cannot happen.

Solution 2 - 100p

Constraints: $N \leq 10^6$

- Quadratic solution is now way too slow but faster ones exist.
- If you for example manage to google your way to the Wikipedia page for the Sylvester-Gallai theorem you will find that an $\mathcal{O}(N \log N)$ solution exists.

G - Ice to Meet You

Problem

Find two students for which a non-educational event may happen or report that such an event cannot happen.

Solution 2 - 100p

Algorithm by Mukhopadhyay & Greene (2012):

- Choose a point p_0 that is a vertex of the convex hull of the given points.
- Construct a line ℓ_0 that passes through p_0 and otherwise stays outside of the convex hull.
- Sort the other given points by the angle they make with p_0, grouping together points that form the same angle.
- If any of the points is alone in its group, then return the ordinary line through that point and p_0.

G - Ice to Meet You

Problem

Find two students for which a non-educational event may happen or report that such an event cannot happen.

Solution 2 - 100p

Algorithm by Mukhopadhyay & Greene (2012):

- For each two consecutive groups of points, in the sorted sequence by their angles, form two lines, each of which passes through the closest point to p_0 in one group and the farthest point from p_0 in the other group.
- For each line ℓ_i in the set of lines formed in this way, find the intersection point of ℓ_i with ℓ_0.
- Return the line ℓ_i whose intersection point with ℓ_0 is the closest to p_0.

G - Ice to Meet You

Problem

Find two students for which a non-educational event may happen or report that such an event cannot happen.

Solution 3 - 100p

- What do the hard cases even look like?

G - Ice to Meet You

Problem

Find two students for which a non-educational event may happen or report that such an event cannot happen.

Solution 3 - 100p

- What do the hard cases even look like?
- Turns out ordinary lines are very common! See "On sets defining few ordinary lines" by Ben Green and Terence Tao (2013).

G - Ice to Meet You

Problem

Find two students for which a non-educational event may happen or report that such an event cannot happen.

Solution 3 - 100p

- What do the hard cases even look like?
- Turns out ordinary lines are very common! See "On sets defining few ordinary lines" by Ben Green and Terence Tao (2013).
- There are at least $N/2$ ordinary lines for large N, meaning probability that a point is ordinary is at least 0.5.

G - Ice to Meet You

Problem

Find two students for which a non-educational event may happen or report that such an event cannot happen.

Solution 3 - 100p

- What do the hard cases even look like?
- Turns out ordinary lines are very common! See "On sets defining few ordinary lines" by Ben Green and Terence Tao (2013).
- There are at least $N/2$ ordinary lines for large N, meaning probability that a point is ordinary is at least 0.5.
- If all points are not colinear, just guess a random point and do a similar line count as in the described $\mathcal{O}(N^2)$ solution. Keep guessing random points until you find an ordinary point.

H - Coin Flip

Problem

Given N weighted coins, find the probability if getting exactly k heads for $0 \leq k \leq N$.

Solution 1 - 15p

H - Coin Flip

Problem

Given N weighted coins, find the probability if getting exactly k heads for $0 \leq k \leq N$.

Solution 1 - 15p

Constraints: $N \leq 2000$

1. Imagine two sets of coins A, B, with A_k chance of getting k heads in A and B_k for B. How do you find the probabilities for $C = A \cup B$?

H - Coin Flip

Problem

Given N weighted coins, find the probability if getting exactly k heads for $0 \leq k \leq N$.

Solution 1 - 15p

Constraints: $N \leq 2000$

1. Imagine two sets of coins A, B, with A_k chance of getting k heads in A and B_k for B. How do you find the probabilities for $C = A \cup B$?
2. Convolution!

$$C_k = \sum_{i=0}^{N} A_i \cdot B_{k-i} \tag{1}$$

H - Coin Flip

Problem

Given N weighted coins, find the probability if getting exactly k heads for $0 \leq k \leq N$.

Solution 1 - 15p

Constraints: $N \leq 2000$

1. Imagine two sets of coins A, B, with A_k chance of getting k heads in A and B_k for B. How do you find the probabilities for $C = A \cup B$?
2. Convolution!

$$C_k = \sum_{i=0}^{N} A_i \cdot B_{k-i} \tag{1}$$

3. For each coin x_i, initialize a list $[1 - x_i/100, x_i/100]$. Then convolve them in some order.

H - Coin Flip

Problem

Given N weighted coins, find the probability if getting exactly k heads for $0 \leq k \leq N$.

Solution 2 - 100p

H - Coin Flip

Problem

Given N weighted coins, find the probability if getting exactly k heads for $0 \leq k \leq N$.

Solution 2 - 100p

Constraints: $N \leq 4 \cdot 10^5$

1. Still convolution, but we need to do it quicker. FFT!

H - Coin Flip

Problem

Given N weighted coins, find the probability if getting exactly k heads for $0 \leq k \leq N$.

Solution 2 - 100p

Constraints: $N \leq 4 \cdot 10^5$

1. Still convolution, but we need to do it quicker. FFT!
2. Also make sure to convolve in an order so that you always convolve lists of similar size, divide and conquer or similar.

H - Coin Flip

Problem

Given N weighted coins, find the probability if getting exactly k heads for $0 \leq k \leq N$.

Solution 2 - 100p

Constraints: $N \leq 4 \cdot 10^5$

1. Still convolution, but we need to do it quicker. FFT!
2. Also make sure to convolve in an order so that you always convolve lists of similar size, divide and conquer or similar.
3. Another option is to do the quadratic convolution, but disregard probabilities that go to 0.

I - Pasta connections

Problem

Given $N \leq 5 \cdot 10^4$ 2D cordinates find an MST (Minimum Spanning Tree)

Solution 1 - 20p

Constraints: $N \leq 2000$

- N is small enough for $\mathcal{O}(n^2)$

I - Pasta connections

Problem

Given $N \leq 5 \cdot 10^4$ 2D cordinates find an MST (Minimum Spanning Tree)

Solution 1 - 20p

Constraints: $N \leq 2000$

- N is small enough for $\mathcal{O}(n^2)$
- Prim's algorithm

I - Pasta connections

Problem

Given $N \leq 5 \cdot 10^4$ 2D cordinates find an MST (Minimum Spanning Tree)

Solution 1 - 20p

Constraints: $N \leq 2000$

- N is small enough for $\mathcal{O}(n^2)$
- Prim's algorithm
- For python, Only add an edge to the queue if it is the shortest edge to the destination node.

I - Pasta connections

Solution 2 - 100p

- Add only the closest point in each direction.

I - Pasta connections

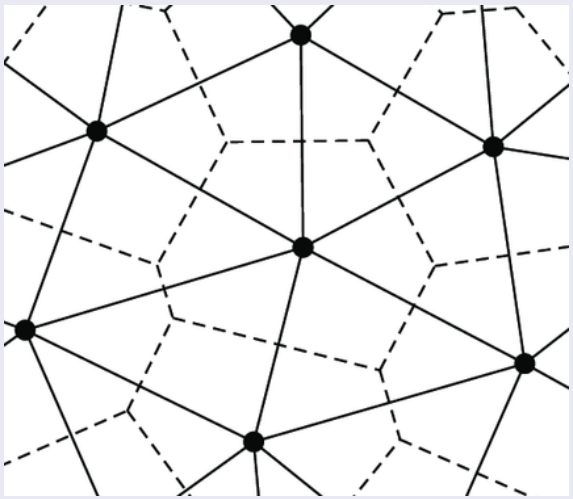
Solution 2 - 100p

- Add only the closest point in each direction.

I - Pasta connections

Solution 2 - 100p

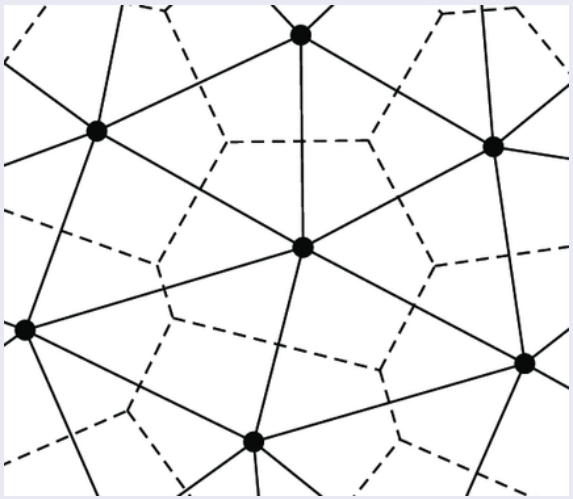
- Add only the closest point in each direction.



I - Pasta connections

Solution 2 - 100p

- Add only the closest point in each direction.



- What defines a close point?

I - Pasta connections

Solution 2 - 100p

- Prims algorithm say: At each step add the point closest to the set of points in the current MST

I - Pasta connections

Solution 2 - 100p

- Prims algorithm say: At each step add the point closest to the set of points in the current MST
- Lets define two points to be "close", iff it is possible to draw a circle for which both points lie on the circumference and no other point lies within the circle.

I - Pasta connections

Solution 2 - 100p

- Prims algorithm say: At each step add the point closest to the set of points in the current MST
- Lets define two points to be "close", iff it is possible to draw a circle for which both points lie on the circumference and no other point lies within the circle.
- Denote the set of all these "close" edges to be $\mathcal{D}$

I - Pasta connections

Solution 2 - 100p

- Prims algorithm say: At each step add the point closest to the set of points in the current MST
- Lets define two points to be "close", iff it is possible to draw a circle for which both points lie on the circumference and no other point lies within the circle.
- Denote the set of all these "close" edges to be $\mathcal{D}$
- lemma: The edges of MST is a subset of the set $\mathcal{D}$, $(MST \subset \mathcal{D})$

Solution 2 - 100p

Prof.

- lets say there exists an edge e (from p_0 to p_1) in the Minimum Spanning Tree that is not in $\mathcal{D}$, $\exists e \in MST, e \notin \mathcal{D}$

Solution 2 - 100p

Prof.

- lets say there exists an edge e (from p_0 to p_1) in the Minimum Spanning Tree that is not in $\mathcal{D}$, $\exists e \in MST, e \notin \mathcal{D}$
- Then at some point prims algorithm will add this edge to the MST.

Solution 2 - 100p

Prof.

- lets say there exists an edge e (from p_0 to p_1) in the Minimum Spanning Tree that is not in $\mathcal{D}$, $\exists e \in MST, e \notin \mathcal{D}$
- Then at some point prims algorithm will add this edge to the MST.
- If prims algorithm adds this edge to the current MST, then this is the shortest edge from the current MST to a point not connected.

Solution 2 - 100p

Prof.

- lets say there exists an edge e (from p_0 to p_1) in the Minimum Spanning Tree that is not in $\mathcal{D}$, $\exists e \in MST, e \notin \mathcal{D}$
- Then at some point prims algorithm will add this edge to the MST.
- If prims algorithm adds this edge to the current MST, then this is the shortest edge from the current MST to a point not connected.
- Since e is not a "close" edge, there must exist a point p_i inside every circle which have both points (p_0, p_1) on the circumference.

Solution 2 - 100p

Prof.

- lets say there exists an edge e (from p_0 to p_1) in the Minimum Spanning Tree that is not in $\mathcal{D}$, $\exists e \in MST, e \notin \mathcal{D}$
- Then at some point prims algorithm will add this edge to the MST.
- If prims algorithm adds this edge to the current MST, then this is the shortest edge from the current MST to a point not connected.
- Since e is not a "close" edge, there must exist a point p_i inside every circle which have both points (p_0, p_1) on the circumference.
- This point p_i must therefore be close to both points on the edge, and then they are to each other.

Solution 2 - 100p

Prof.

- lets say there exists an edge e (from p_0 to p_1) in the Minimum Spanning Tree that is not in $\mathcal{D}$, $\exists e \in MST, e \notin \mathcal{D}$
- Then at some point prims algorithm will add this edge to the MST.
- If prims algorithm adds this edge to the current MST, then this is the shortest edge from the current MST to a point not connected.
- Since e is not a "close" edge, there must exist a point p_i inside every circle which have both points (p_0, p_1) on the circumference.
- This point p_i must therefore be close to both points on the edge, and then they are to each other.
- if $p_i \in MST$ then the edge $p_i \to p_1$ would be added

Solution 2 - 100p

Prof.

- lets say there exists an edge e (from p_0 to p_1) in the Minimum Spanning Tree that is not in $\mathcal{D}$, $\exists e \in MST, e \notin \mathcal{D}$
- Then at some point prims algorithm will add this edge to the MST.
- If prims algorithm adds this edge to the current MST, then this is the shortest edge from the current MST to a point not connected.
- Since e is not a "close" edge, there must exist a point p_i inside every circle which have both points (p_0, p_1) on the circumference.
- This point p_i must therefore be close to both points on the edge, and then they are to each other.
- if $p_i \in MST$ then the edge $p_i \to p_1$ would be added
- if $p_i \notin MST$ Then the edge $p_0 \to p_i$ would be added

I - Pasta connections

Solution 2 - 100p

- Now find all the "close" edges and only consider those in the MST.

I - Pasta connections

Solution 2 - 100p

- Now find all the "close" edges and only consider those in the MST.
- Our definition of a "close" edge is precisely the definition on a Delaunay triangulation, which can be found in $\mathcal{O}(n \log n)$

I - Pasta connections

Solution 2 - 100p

- Now find all the "close" edges and only consider those in the MST.
- Our definition of a "close" edge is precisely the definition on a Delaunay triangulation, which can be found in $\mathcal{O}(n \log n)$
- Apply prims or kruskals algorithm to find the MST from this subset of all edges

I - Pasta connections

Solution 2 - 100p

- Now find all the "close" edges and only consider those in the MST.
- Our definition of a "close" edge is precisely the definition on a Delaunay triangulation, which can be found in $\mathcal{O}(n \log n)$
- Apply prims or kruskals algorithm to find the MST from this subset of all edges
- The number of edges in a Delaunay triangulation is $3n - 6$.

J - Husdrömmar

Problem

Minimize $x^T F x + s^T x$
s.t. $Ax \geq b$.

Solution 1 - 10p

J - Husdrömmar

Problem

Minimize $x^T F x + s^T x$
s.t. $Ax \geq b$.

Solution 1 - 10p

Constraint: $N \leq 50$. Use a standard solver, can find it on Google.

J - Husdrömmar

Problem

Minimize $x^T F x + s^T x$
s.t. $Ax \geq b$.

Solution 1 - 10p

Constraint: $N \leq 50$. Use a standard solver, can find it on Google.

Solution 2 - 100p

J - Husdrömmar

Problem

Minimize $x^T F x + s^T x$
s.t. $Ax \geq b$.

Solution 1 - 10p

Constraint: $N \leq 50$. Use a standard solver, can find it on Google.

Solution 2 - 100p

The solution consists of two steps:

1. Find the analytic solution of the problem using optimization and the hints in the statement.

J - Husdrömmar

Problem

Minimize $x^T F x + s^T x$
s.t. $Ax \geq b$.

Solution 1 - 10p

Constraint: $N \leq 50$. Use a standard solver, can find it on Google.

Solution 2 - 100p

The solution consists of two steps:

1. Find the analytic solution of the problem using optimization and the hints in the statement.
2. Use an efficient algorithm to actually compute the analytic solution.

Analytic solution

Claim

F is positive definite, i.e. $x^T F x > 0 \quad \forall x \in \mathbb{R}^N \setminus \{0\}$.
If this is true, it means that the problem is strictly convex and there is a unique minimizer.

Proof

Analytic solution

Claim

F is positive definite, i.e. $x^T F x > 0 \quad \forall x \in \mathbb{R}^N \setminus \{0\}$.
If this is true, it means that the problem is strictly convex and there is a unique minimizer.

Proof

Statement says that $F = F_0 F_0^T + I$. Now just put x on it:

$$x^T F x = x^T (F_0 F_0^T + I) x \tag{2}$$

(6)

Analytic solution

Claim

F is positive definite, i.e. $x^T F x > 0 \quad \forall x \in \mathbb{R}^N \setminus \{0\}$.
If this is true, it means that the problem is strictly convex and there is a unique minimizer.

Proof

Statement says that $F = F_0 F_0^T + I$. Now just put x on it:

$$x^T F x = x^T (F_0 F_0^T + I) x \tag{2}$$
$$= x^T F_0 F_0^T x + x^T x \tag{3}$$

$$\tag{6}$$

Analytic solution

Claim

F is positive definite, i.e. $x^T F x > 0 \quad \forall x \in \mathbb{R}^N \setminus \{0\}$.
If this is true, it means that the problem is strictly convex and there is a unique minimizer.

Proof

Statement says that $F = F_0 F_0^T + I$. Now just put x on it:

$$\begin{aligned} x^T F x &= x^T (F_0 F_0^T + I) x && (2) \\ &= x^T F_0 F_0^T x + x^T x && (3) \\ &= (F_0^T x)^T (F_0^T x) + x^T x && (4) \end{aligned}$$

$$(6)$$

Analytic solution

Claim

F is positive definite, i.e. $x^T F x > 0 \quad \forall x \in \mathbb{R}^N \setminus \{0\}$.
If this is true, it means that the problem is strictly convex and there is a unique minimizer.

Proof

Statement says that $F = F_0 F_0^T + I$. Now just put x on it:

$$
\begin{aligned}
x^T F x &= x^T (F_0 F_0^T + I) x && (2)\\
&= x^T F_0 F_0^T x + x^T x && (3)\\
&= (F_0^T x)^T (F_0^T x) + x^T x && (4)\\
&= \|F_0^T x\|^2 + \|x\|^2 && (5)\\
& && (6)
\end{aligned}
$$

Analytic solution

Claim

F is positive definite, i.e. $x^T F x > 0 \quad \forall x \in \mathbb{R}^N \setminus \{0\}$.
If this is true, it means that the problem is strictly convex and there is a unique minimizer.

Proof

Statement says that $F = F_0 F_0^T + I$. Now just put x on it:

$$
\begin{aligned}
x^T F x &= x^T (F_0 F_0^T + I) x && (2)\\
&= x^T F_0 F_0^T x + x^T x && (3)\\
&= (F_0^T x)^T (F_0^T x) + x^T x && (4)\\
&= \|F_0^T x\|^2 + \|x\|^2 && (5)\\
&> 0 \quad \forall x \in \mathbb{R}^N \setminus \{0\} && (6)
\end{aligned}
$$

Analytic solution

Problem

Minimize $x^T F x + s^T x$
s.t. $Ax \geq b$.

Lagrangian

1. $Ax \geq b \iff b - Ax \leq 0$.

Analytic solution

Problem

Minimize $x^T F x + s^T x$
s.t. $Ax \geq b$.

Lagrangian

1. $Ax \geq b \iff b - Ax \leq 0$.
2. The problem can now be rewritten as

$$\min_x \quad x^T F x + s^T x \tag{7}$$

$$\text{Subject to} \quad b - Ax \leq 0 \tag{8}$$

Analytic solution

Problem

Minimize $x^T F x + s^T x$
s.t. $Ax \geq b$.

Lagrangian

1. $Ax \geq b \iff b - Ax \leq 0$.
2. The problem can now be rewritten as

$$\min_x \quad x^T F x + s^T x \tag{7}$$

$$\text{Subject to} \quad b - Ax \leq 0 \tag{8}$$

3. We define the Lagrangian as

$$\mathcal{L}(x, \lambda) = x^T F x + s^T x + \lambda^T (b - Ax) \tag{9}$$

Analytic solution

Lagrangian

$$\mathcal{L}(x, \lambda) = x^T F x + s^T x + \lambda^T (b - Ax) \tag{10}$$

Duality

Analytic solution

Lagrangian

$$\mathcal{L}(x, \lambda) = x^T F x + s^T x + \lambda^T (b - Ax) \tag{10}$$

Duality

1. Dual function:

$$g(\lambda) = \inf_x \mathcal{L}(x, \lambda) \tag{11}$$

Analytic solution

Lagrangian

$$\mathcal{L}(x, \lambda) = x^T F x + s^T x + \lambda^T (b - Ax) \tag{10}$$

Duality

1. Dual function:
$$g(\lambda) = \inf_x \mathcal{L}(x, \lambda) \tag{11}$$
2. F is positive definite $\implies$ $\mathcal{L}(x, \lambda)$ has a unique minimizer x, where $\nabla_x \mathcal{L}(x, \lambda) = 0$.

Analytic solution

Lagrangian

$$\mathcal{L}(x, \lambda) = x^T F x + s^T x + \lambda^T (b - Ax) \tag{10}$$

Duality

1. Dual function:
$$g(\lambda) = \inf_x \mathcal{L}(x, \lambda) \tag{11}$$
2. F is positive definite $\implies$ $\mathcal{L}(x, \lambda)$ has a unique minimizer x, where $\nabla_x \mathcal{L}(x, \lambda) = 0$.
3. $\nabla_x \mathcal{L}(x, \lambda) = 2Fx + s - A^T \lambda$.

Analytic solution

Lagrangian

$$\mathcal{L}(x, \lambda) = x^T F x + s^T x + \lambda^T (b - Ax) \tag{10}$$

Duality

1. Dual function:
$$g(\lambda) = \inf_x \mathcal{L}(x, \lambda) \tag{11}$$
2. F is positive definite $\implies$ $\mathcal{L}(x, \lambda)$ has a unique minimizer x, where $\nabla_x \mathcal{L}(x, \lambda) = 0$.
3. $\nabla_x \mathcal{L}(x, \lambda) = 2Fx + s - A^T \lambda$.
4. $2Fx + s - A^T \lambda = 0 \iff x = \frac{1}{2} F^{-1}(A^T \lambda - s)$

Analytic solution

Lagrangian

$$\mathcal{L}(x, \lambda) = x^T F x + s^T x + \lambda^T (b - Ax) \tag{10}$$

Duality

1. Dual function:
$$g(\lambda) = \inf_x \mathcal{L}(x, \lambda) \tag{11}$$
2. F is positive definite $\implies$ $\mathcal{L}(x, \lambda)$ has a unique minimizer x, where $\nabla_x \mathcal{L}(x, \lambda) = 0$.
3. $\nabla_x \mathcal{L}(x, \lambda) = 2Fx + s - A^T \lambda$.
4. $2Fx + s - A^T \lambda = 0 \iff x = \frac{1}{2} F^{-1}(A^T \lambda - s)$
5. Denote $\hat{x}(\lambda) = \frac{1}{2} F^{-1}(A\lambda - s)$ and insert into $\mathcal{L}(x, \lambda)$.

Analytic solution

Lagrangian

$$\mathcal{L}(x, \lambda) = x^T F x + s^T x + \lambda^T (b - Ax) \tag{12}$$

Duality

Analytic solution

Lagrangian

$$\mathcal{L}(x, \lambda) = x^T F x + s^T x + \lambda^T (b - Ax) \tag{12}$$

Duality

Dual function:

$$g(\lambda) = \inf_x \mathcal{L}(x, \lambda) \tag{13}$$

(16)

Analytic solution

Lagrangian

$$\mathcal{L}(x, \lambda) = x^T F x + s^T x + \lambda^T (b - Ax) \tag{12}$$

Duality

Dual function:

$$g(\lambda) = \inf_x \mathcal{L}(x, \lambda) \tag{13}$$

$$= \mathcal{L}(\hat{x}(\lambda), \lambda) \tag{14}$$

$$\tag{16}$$

Analytic solution

Lagrangian

$$\mathcal{L}(x, \lambda) = x^T F x + s^T x + \lambda^T (b - Ax) \tag{12}$$

Duality

Dual function:

$$g(\lambda) = \inf_x \mathcal{L}(x, \lambda) \tag{13}$$

$$= \mathcal{L}(\hat{x}(\lambda), \lambda) \tag{14}$$

$$= \text{lots of stuff...} \tag{15}$$

$$\tag{16}$$

Analytic solution

Lagrangian

$$\mathcal{L}(x, \lambda) = x^T F x + s^T x + \lambda^T (b - Ax) \tag{12}$$

Duality

Dual function:

$$\begin{aligned} g(\lambda) &= \inf_x \mathcal{L}(x, \lambda) && (13)\\ &= \mathcal{L}(\hat{x}(\lambda), \lambda) && (14)\\ &= \text{lots of stuff}\ldots && (15)\\ &= -\frac{1}{4}\lambda^T A F^{-1} A \lambda + (b + \frac{1}{2} A F^{-1} s)^T \lambda - \frac{1}{4} s^T F^{-1} s && (16) \end{aligned}$$

Analytic solution

Dual function

$$g(\lambda) = -\frac{1}{4}\lambda^T A F^{-1} A\lambda + (b + \frac{1}{2}AF^{-1}s)^T\lambda - \frac{1}{4}s^T F^{-1}s \qquad (17)$$

Dual problem

Analytic solution

Dual function

$$g(\lambda) = -\frac{1}{4}\lambda^T A F^{-1} A\lambda + (b + \frac{1}{2}AF^{-1}s)^T\lambda - \frac{1}{4}s^T F^{-1}s \qquad (17)$$

Dual problem

1. Formulate the dual problem: $\max_{\lambda \geq 0} g(\lambda)$.

Analytic solution

Dual function

$$g(\lambda) = -\frac{1}{4}\lambda^T A F^{-1} A\lambda + (b + \frac{1}{2}AF^{-1}s)^T\lambda - \frac{1}{4}s^T F^{-1}s \qquad (17)$$

Dual problem

1. Formulate the dual problem: $\max_{\lambda \geq 0} g(\lambda)$.
2. Because the primal (original) problem was convex, this problem has the same optimal value! Maybe we can try to solve this instead.

Analytic solution

Dual function

$$g(\lambda) = -\frac{1}{4}\lambda^T A F^{-1} A\lambda + (b + \frac{1}{2}AF^{-1}s)^T\lambda - \frac{1}{4}s^T F^{-1}s \tag{17}$$

Dual problem

1. Formulate the dual problem: $\max_{\lambda \geq 0} g(\lambda)$.
2. Because the primal (original) problem was convex, this problem has the same optimal value! Maybe we can try to solve this instead.
3. It can be shown that $g(\lambda)$ is strictly concave. If we didn't have the constraint $\lambda \geq 0$, we could just set $\nabla g(\lambda) = 0$ and solve for λ.

Analytic solution

Dual function

$$g(\lambda) = -\frac{1}{4}\lambda^T A F^{-1} A\lambda + (b + \frac{1}{2}AF^{-1}s)^T\lambda - \frac{1}{4}s^T F^{-1}s \tag{17}$$

Dual problem

1. Formulate the dual problem: $\max_{\lambda \geq 0} g(\lambda)$.
2. Because the primal (original) problem was convex, this problem has the same optimal value! Maybe we can try to solve this instead.
3. It can be shown that $g(\lambda)$ is strictly concave. If we didn't have the constraint $\lambda \geq 0$, we could just set $\nabla g(\lambda) = 0$ and solve for λ.
4. Let's just try to solve for λ anyway and see what happens!

Analytic solution

Dual function

$$g(\lambda) = -\frac{1}{4}\lambda^T A F^{-1} A\lambda + (b + \frac{1}{2}AF^{-1}s)^T\lambda - \frac{1}{4}s^T F^{-1}s \tag{18}$$

Solving for λ

Analytic solution

Dual function

$$g(\lambda) = -\frac{1}{4}\lambda^T A F^{-1} A\lambda + (b + \frac{1}{2}AF^{-1}s)^T\lambda - \frac{1}{4}s^T F^{-1}s \tag{18}$$

Solving for λ

1. $\nabla g(\lambda) = -\frac{1}{2}AF^{-1}A\lambda + b + \frac{1}{2}AF^{-1}s$

Analytic solution

Dual function

$$g(\lambda) = -\frac{1}{4}\lambda^T A F^{-1} A\lambda + (b + \frac{1}{2}AF^{-1}s)^T\lambda - \frac{1}{4}s^T F^{-1}s \qquad (18)$$

Solving for λ

1. $\nabla g(\lambda) = -\frac{1}{2}AF^{-1}A\lambda + b + \frac{1}{2}AF^{-1}s$
2. $-\frac{1}{2}AF^{-1}A\lambda + b + \frac{1}{2}AF^{-1}s = 0 \iff \lambda = 2A^{-1}FA^{-1}b + A^{-1}s.$

Analytic solution

Dual function

$$g(\lambda) = -\frac{1}{4}\lambda^T A F^{-1} A\lambda + (b + \frac{1}{2}AF^{-1}s)^T\lambda - \frac{1}{4}s^T F^{-1}s \qquad (18)$$

Solving for λ

1. $\nabla g(\lambda) = -\frac{1}{2}AF^{-1}A\lambda + b + \frac{1}{2}AF^{-1}s$
2. $-\frac{1}{2}AF^{-1}A\lambda + b + \frac{1}{2}AF^{-1}s = 0 \iff \lambda = 2A^{-1}FA^{-1}b + A^{-1}s.$
3. Is this λ positive? The statement says that $b = \frac{1}{2}AF^{-1}(Ac - s)$, insert it into the formula.

Analytic solution

Dual function

$$g(\lambda) = -\frac{1}{4}\lambda^T A F^{-1} A\lambda + (b + \frac{1}{2}AF^{-1}s)^T\lambda - \frac{1}{4}s^T F^{-1}s \qquad (18)$$

Solving for λ

1. $\nabla g(\lambda) = -\frac{1}{2}AF^{-1}A\lambda + b + \frac{1}{2}AF^{-1}s$
2. $-\frac{1}{2}AF^{-1}A\lambda + b + \frac{1}{2}AF^{-1}s = 0 \iff \lambda = 2A^{-1}FA^{-1}b + A^{-1}s.$
3. Is this λ positive? The statement says that $b = \frac{1}{2}AF^{-1}(Ac - s)$, insert it into the formula.
4. $\lambda = 2A^{-1}FA^{-1}b + A^{-1}s = \text{stuff} \ldots = c.$ And $1 \leq c \leq 3.$ Perfect!

Analytic solution

Dual function

$$g(\lambda) = -\frac{1}{4}\lambda^T A F^{-1} A\lambda + (b + \frac{1}{2}AF^{-1}s)^T\lambda - \frac{1}{4}s^T F^{-1}s \tag{19}$$

Optimal value

1. $\lambda = 2A^{-1}FA^{-1}b + A^{-1}s$ maximizes $g(\lambda)$.

Analytic solution

Dual function

$$g(\lambda) = -\frac{1}{4}\lambda^T A F^{-1} A \lambda + (b + \frac{1}{2} A F^{-1} s)^T \lambda - \frac{1}{4} s^T F^{-1} s \tag{19}$$

Optimal value

1. $\lambda = 2A^{-1}FA^{-1}b + A^{-1}s$ maximizes $g(\lambda)$.
2. $g(2A^{-1}FA^{-1}b + A^{-1}s) = \text{stuff} \ldots = b^T A^{-1} F A^{-1} b + s^T A^{-1} b.$

Analytic solution

Dual function

$$g(\lambda) = -\frac{1}{4}\lambda^T A F^{-1} A \lambda + (b + \frac{1}{2} A F^{-1} s)^T \lambda - \frac{1}{4} s^T F^{-1} s \tag{19}$$

Optimal value

1. $\lambda = 2A^{-1}FA^{-1}b + A^{-1}s$ maximizes $g(\lambda)$.
2. $g(2A^{-1}FA^{-1}b + A^{-1}s) = \text{stuff} \ldots = b^T A^{-1} F A^{-1} b + s^T A^{-1} b$.
3. If you want to you can do some fancy math to bring back the solution of the primal problem. Or you can just "pattern match" it: What x gives you $x^T F x + s^T x = b^T A^{-1} F A^{-1} b + s^T A^{-1} b$?

Analytic solution

Dual function

$$g(\lambda) = -\frac{1}{4}\lambda^T A F^{-1} A \lambda + (b + \frac{1}{2} A F^{-1} s)^T \lambda - \frac{1}{4} s^T F^{-1} s \tag{19}$$

Optimal value

1. $\lambda = 2A^{-1}FA^{-1}b + A^{-1}s$ maximizes $g(\lambda)$.
2. $g(2A^{-1}FA^{-1}b + A^{-1}s) = \text{stuff} \ldots = b^T A^{-1} F A^{-1} b + s^T A^{-1} b$.
3. If you want to you can do some fancy math to bring back the solution of the primal problem. Or you can just "pattern match" it: What x gives you $x^T F x + s^T x = b^T A^{-1} F A^{-1} b + s^T A^{-1} b$?
4. The answer is $x = A^{-1}b$. A bit anticlimactic after all these slides...

Computing the solution

Optimal solution

$$x = A^{-1}b \tag{20}$$

How to compute it quickly?

Computing the solution

Optimal solution

$$x = A^{-1}b \tag{20}$$

How to compute it quickly?

1. Kactl's linear solver is too slow, $\mathcal{O}(N^3)$.

Computing the solution

Optimal solution

$$x = A^{-1}b \tag{20}$$

How to compute it quickly?

1. Kactl's linear solver is too slow, $\mathcal{O}(N^3)$.
2. Recall that $A = A_0 A_0^T + I$. Similarly to F, A is also positive definite! Can solve $Ax = b$ by using the Cholesky factorization $A = LL^T$. Yields $\mathcal{O}(N^3)$ with very good constant factor, but it is still too slow.

Computing the solution

Optimal solution

$$x = A^{-1}b \tag{20}$$

How to compute it quickly?

1. Kactl's linear solver is too slow, $\mathcal{O}(N^3)$.
2. Recall that $A = A_0 A_0^T + I$. Similarly to F, A is also positive definite! Can solve $Ax = b$ by using the Cholesky factorization $A = LL^T$. Yields $\mathcal{O}(N^3)$ with very good constant factor, but it is still too slow.
3. The conjugate gradient method solves $Ax = b$ for positive definite A, exactly what we want! Iterative method, each step takes $\mathcal{O}(N^2)$, and it needs at most N steps. Maybe it can converge quickly?

Computing the solution

Convergence of conjugate gradient

1. Define the error in step k as $e_k = x_k - A^{-1}b$, and start with $x_0 = 0$.

Computing the solution

Convergence of conjugate gradient

1. Define the error in step k as $e_k = x_k - A^{-1}b$, and start with $x_0 = 0$.
2. $k = \frac{1}{2}\sqrt{\kappa(A)}\log(2|b^T A^{-1}b|\epsilon^{-1})$ iterations is enough to reduce $|e_k^T A e_k|$ to ϵ, for any $\epsilon > 0$.

Computing the solution

Convergence of conjugate gradient

1. Define the error in step k as $e_k = x_k - A^{-1}b$, and start with $x_0 = 0$.
2. $k = \frac{1}{2}\sqrt{\kappa(A)}\log(2|b^T A^{-1} b|\epsilon^{-1})$ iterations is enough to reduce $|e_k^T A e_k|$ to ϵ, for any $\epsilon > 0$.
3. But what is $\kappa(A)$?

Computing the solution

Random matrix theory

1. A_0 has standard deviation $\frac{1}{\sqrt{N}}$. It would be the same as letting $A_0 = \frac{1}{\sqrt{N}} X$ where X has standard deviation 1.

Computing the solution

Random matrix theory

1. A_0 has standard deviation $\frac{1}{\sqrt{N}}$. It would be the same as letting $A_0 = \frac{1}{\sqrt{N}} X$ where X has standard deviation 1.
2. Let $Y = A_0 A_0^T = (\frac{1}{\sqrt{N}} X)(\frac{1}{\sqrt{N}} X)^T = \frac{1}{N} X X^T$.

Computing the solution

Random matrix theory

1. A_0 has standard deviation $\frac{1}{\sqrt{N}}$. It would be the same as letting $A_0 = \frac{1}{\sqrt{N}} X$ where X has standard deviation 1.
2. Let $Y = A_0 A_0^T = (\frac{1}{\sqrt{N}} X)(\frac{1}{\sqrt{N}} X)^T = \frac{1}{N} X X^T$.
3. The Marchenko–Pastur law now tells us that the probability density function of the eigenvalues of Y as $N \to \infty$ is:

$$\nu(x) = \begin{cases} \frac{\sqrt{x(4-x)}}{2\pi x}, \text{ if } 0 < x < 4 \\ 0, \text{ otherwise} \end{cases} . \tag{21}$$

Computing the solution

Random matrix theory

1. A_0 has standard deviation $\frac{1}{\sqrt{N}}$. It would be the same as letting $A_0 = \frac{1}{\sqrt{N}} X$ where X has standard deviation 1.
2. Let $Y = A_0 A_0^T = (\frac{1}{\sqrt{N}} X)(\frac{1}{\sqrt{N}} X)^T = \frac{1}{N} X X^T$.
3. The Marchenko–Pastur law now tells us that the probability density function of the eigenvalues of Y as $N \to \infty$ is:

$$\nu(x) = \begin{cases} \frac{\sqrt{x(4-x)}}{2\pi x}, \text{ if } 0 < x < 4 \\ 0, \text{ otherwise} \end{cases} . \tag{21}$$

4. $A = A_0 A_0^T + I = Y + I$, so its eigenvalues are just shifted up by 1, i.e. they range from 1 to 5.

Computing the solution

Random matrix theory

1. A_0 has standard deviation $\frac{1}{\sqrt{N}}$. It would be the same as letting $A_0 = \frac{1}{\sqrt{N}} X$ where X has standard deviation 1.
2. Let $Y = A_0 A_0^T = (\frac{1}{\sqrt{N}} X)(\frac{1}{\sqrt{N}} X)^T = \frac{1}{N} X X^T$.
3. The Marchenko–Pastur law now tells us that the probability density function of the eigenvalues of Y as $N \to \infty$ is:

$$\nu(x) = \begin{cases} \frac{\sqrt{x(4-x)}}{2\pi x}, \text{ if } 0 < x < 4 \\ 0, \text{ otherwise} \end{cases} . \tag{21}$$

4. $A = A_0 A_0^T + I = Y + I$, so its eigenvalues are just shifted up by 1, i.e. they range from 1 to 5.
5. A symmetric $\implies \kappa(A) = \frac{\lambda_{max}(A)}{\lambda_{min}(A)} \leq \frac{5}{1} = 5$.

Computing the solution

Random matrix theory

1. A_0 has standard deviation $\frac{1}{\sqrt{N}}$. It would be the same as letting $A_0 = \frac{1}{\sqrt{N}}X$ where X has standard deviation 1.
2. Let $Y = A_0 A_0^T = (\frac{1}{\sqrt{N}}X)(\frac{1}{\sqrt{N}}X)^T = \frac{1}{N}XX^T$.
3. The Marchenko–Pastur law now tells us that the probability density function of the eigenvalues of Y as $N \to \infty$ is:

$$\nu(x) = \begin{cases} \frac{\sqrt{x(4-x)}}{2\pi x}, \text{ if } 0 < x < 4 \\ 0, \text{ otherwise} \end{cases} . \tag{21}$$

4. $A = A_0 A_0^T + I = Y + I$, so its eigenvalues are just shifted up by 1, i.e. they range from 1 to 5.
5. A symmetric $\implies \kappa(A) = \frac{\lambda_{max}(A)}{\lambda_{min}(A)} \leq \frac{5}{1} = 5$.
6. Alternatively you can just generate many A and check $\kappa(A)$.

Computing the solution

Convergence of conjugate gradient

1. $k = \frac{1}{2}\sqrt{\kappa(A)}\log(2|b^T A^{-1} b|\epsilon^{-1})$ iterations were needed. Insert $\kappa(A) = 5$.

Computing the solution

Convergence of conjugate gradient

1. $k = \frac{1}{2}\sqrt{\kappa(A)}\log(2|b^T A^{-1} b|\epsilon^{-1})$ iterations were needed. Insert $\kappa(A) = 5$.
2. $k = \frac{\sqrt{5}}{2}\log(2|b^T A^{-1} b|\epsilon^{-1})$

Computing the solution

Convergence of conjugate gradient

1. $k = \frac{1}{2}\sqrt{\kappa(A)}\log(2|b^T A^{-1} b|\epsilon^{-1})$ iterations were needed. Insert $\kappa(A) = 5$.
2. $k = \frac{\sqrt{5}}{2}\log(2|b^T A^{-1} b|\epsilon^{-1})$
3. With $\epsilon = 10^{-10}$ you get around 30 iterations maximum but in practice you don't need that much.